

Assignment II: vehicle state estimation

Implement the EKF example with code:

Download *vehicle_state_estimation.zip*

- Predict with motion model (IMU + Wheel)
complete the function *EkfPredict()*

```
void EkfPredict(State& state, const double &time, const double &velocity, const double &yaw_rate) {  
    // printf("time %lf, velocity %lf, yaw_rate %lf \n", time, velocity, yaw_rate);  
    // YOUR_CODE_HERE  
    // todo: implement the EkfPredict function  
  
    // printf("after predict x: %lf, y: %lf, yaw: %lf \n", state.x, state.y, state.yaw);  
}
```

- Update with measurement model (GPS)
complete the function *EkfUpdate()*

```
void EkfPredict(State& state, const double &time, const double &velocity, const double &yaw_rate) {  
    // printf("time %lf, velocity %lf, yaw_rate %lf \n", time, velocity, yaw_rate);  
    // YOUR_CODE_HERE  
    // todo: implement the EkfPredict function  
  
    // printf("after predict x: %lf, y: %lf, yaw: %lf \n", state.x, state.y, state.yaw);  
}
```

- Build the project and run the node, visualize the estimation trajectory in RVIZ

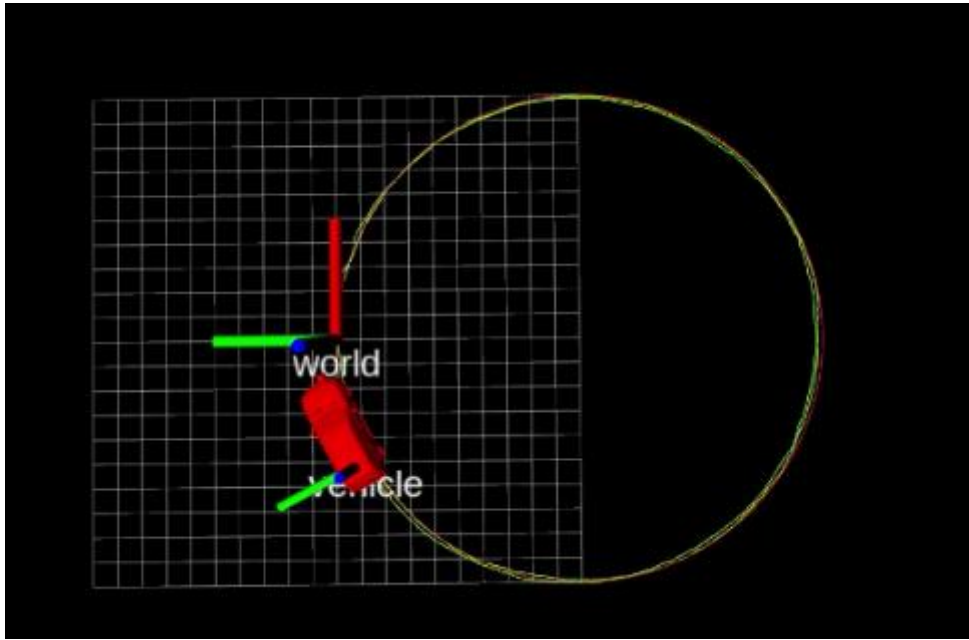
```
cd ~/catkin_ws/src  
catkin_make
```

```
roscore
```

```
source ~/catkin_ws/devel/setup.bash  
roslaunch vehicle_state_estimation vehicle_state_estimation
```

```
source ~/catkin_ws/devel/setup.bash  
roslaunch rviz rviz -d ~/catkin_ws/src/vehicle_state_estimation/state_estimation.rviz
```

- Expected result:
(red(ground truth) and green path(estimation result) are closed!)



Submission:

Write a report to explain how to complete the EKF function, and attach your source code and the screenshot of your estimation in RVIZ.